FINDINGS DOSSIER

Coturn: struct-field bounds in the internal proofs

Findings dossier, source excerpts and paper reuse package

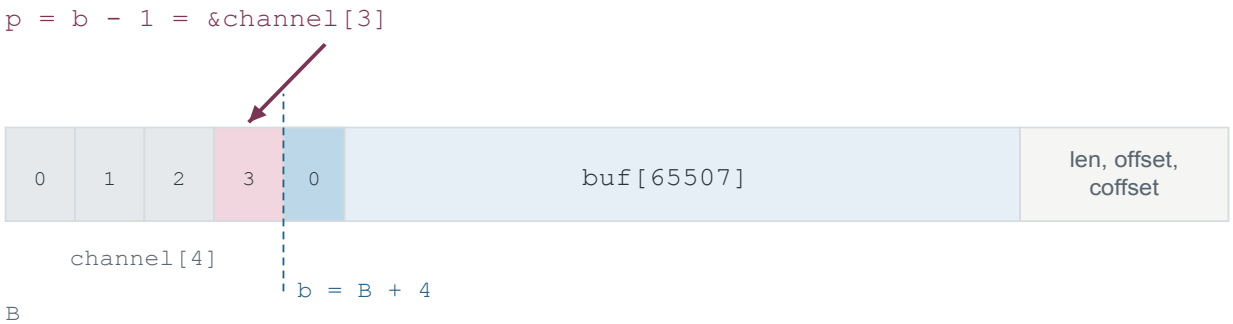
1 November 2026

Coturn is a TURN and STUN server implementation. The reviewed target is FuzzStunClient. The ASan and ASan+UBSan coturn proof cases have machine-checked models satisfying the complete original internal invariant while a buffer-derived write address is one byte before the `buf` field. The address is the storage of the preceding `channel[3]` field, within the same `stun_buffer`. See the pinned project README under upstream/coturn/README.md.

Established: non-entailment of the field lower bound by the selected internal invariants. Input reachability, attacker control and downstream corruption are unestablished.

Finding	Evidence level	Conclusion
Coturn ASan	Full original gamma2 witness checked in Rocq 9.2	The selected case admits buf - 1.
Coturn ASan+UBSan	Full original gamma2 witness checked in Rocq 9.2	The selected case admits buf - 1.
Coturn plain	Source, binary and decoded-certificate analysis; Python consistency checks	Matching local case found; no new full-gamma Rocq replay for this variant.
bzip2 candidate	Manual source/proof audit and RV64 layout checks	No omitted field-bound example identified among 24 core inline arrays.

The report freezes proof repository commit `a6839bb9e433b34c0d0710df5280194528877139`, coturn source `70ca74e0bd8300e22a179e9a7beead862763ca5c`, and the paper at `1c1b1e742b1b80fa975a976f6e94ff7844ea07dd`. These were the checked repository HEADs or pinned source revision during the investigation. Paper citations use the committed snapshot preserved in `upstream/paper/main.tex`; concurrent local manuscript edits remain untouched.



First byte targets channel[3]; second byte is at buf[0].

The address relation in the audited RV64 layout. The drawing is schematic; the two arrays are not drawn to scale.

1. The C objects and the static call context

The declaration has two distinct array fields. `STUN_CHANNEL_HEADER_LENGTH` is 4 and `STUN_BUFFER_SIZE` is 65507. [C1] [C2] [C3]

stun_buffer declaration [C1]

```
45  typedef struct _stun_buffer {
46    uint8_t channel[STUN_CHANNEL_HEADER_LENGTH];
47    uint8_t buf[STUN_BUFFER_SIZE];
48    size_t len;
49    uint16_t offset;
50    uint8_t coffset;
51  } stun_buffer;
```

In the audited RV64 layout, `channel` starts at offset 0 and `buf` at offset 4. The remaining offsets are `len` at 65512, `offset` at 65520 and `coffset` at 65522; the whole struct has size 65528 and alignment 8. `checks/layout-rv64.txt` records an independent layout-only Clang check using matching scalar types and declarations.

Struct wrapper passes its buf field [C4]

```
181  uint16_t stun_set_channel_bind_request(stun_buffer *buf, const ioa_addr *peer_addr, uint16_t
     channel_number) {
182
183    return stun_set_channel_bind_request_str(buf->buf, &(buf->len), peer_addr, channel_number);
184  }
```

For the reviewed certificate context, the wrapper passes its array field `buf->buf` and the address of its length field into the `_str` API. The static source call chain continues through `stun_set_channel_bind_request_str`, `stun_attr_add_addr_str`, and `stun_attr_add_str`. The address builder makes the final call at line 1729. This identifies the source field and call context; it is not an input-reachability argument. [C5] [C6]

Signed message-length result and -1 failure [C7]

```
523  int stun_get_command_message_len_str(const uint8_t *buf, size_t len) {
524    if (len < STUN_HEADER_LENGTH) {
525      return -1;
526    }
527
528    /* Validate the size the buffer claims to be */
529    const size_t bufLen = (size_t)(turn_read_u16(buf + 2) + STUN_HEADER_LENGTH);
530    if (bufLen > len) {
531      return -1;
532    }
533
534    return bufLen;
535  }
```

The signed result is either a validated message length or the failure value -1. Identifying that error value does not establish that this caller can produce it. In particular, the abstract witness is not a proof that this helper returns -1 on the witness memory.

2. The access and its missing field-bound obligation

The following is the complete body of `stun_attr_add_str`, so the argument normalization, length check, padding and writes remain visible. [C8]

Complete stun_attr_add_str body [C8]

```
1666  bool stun_attr_add_str(uint8_t *buf, size_t *len, uint16_t attr, const uint8_t *avalue, int alen)
      {
1667    if (alen < 0) {
1668      alen = 0;
1669    }
1670    uint8_t tmp[1];
1671    if (!avalue) {
1672      alen = 0;
1673      avalue = tmp;
1674    }
1675    const int clen = stun_get_command_message_len_str(buf, *len);
1676    int newlen = clen + 4 + alen;
1677    const int newlenrem4 = newlen & 0x00000003;
1678    int paddinglen = 0;
1679    if (newlenrem4) {
1680      paddinglen = 4 - newlenrem4;
1681      newlen = newlen + paddinglen;
1682    }
1683
1684    if (newlen >= MAX_STUN_MESSAGE_SIZE) {
1685      return false;
1686    }
1687
1688    uint8_t *attr_start = buf + clen;
1689
1690    stun_set_command_message_len_str(buf, newlen);
1691    *len = newlen;
1692
1693    turn_write_u16(attr_start, attr);
1694    turn_write_u16(attr_start + 2, (uint16_t)alen);
1695    if (alen > 0) {
1696      memcpy(attr_start + 4, avalue, alen);
1697    }
1698
1699    // Write 0 padding to not leak data
1700    memset(attr_start + 4 + alen, 0, paddinglen);
1701
1702    return true;
1703  }
```

Two-byte write helper [C9]

```
 145  static inline void turn_write_u16(void *field, uint16_t v) {
 146    const uint16_t raw = nswap16(v);
 147    memcpy(field, &raw, sizeof(raw));
 148  }
```

The helper writes two bytes. Let B be the struct base, b = B + 4 the start of `buf`, and p the write destination. In the admitted case, p = b - 1 = B + 3, the storage of `channel[3]`; the second byte is at `buf[0]`. The capacity check on `newlen` is an upper-bound check and does not by itself prove `clen >= 0`.

```
Required for the two-byte write:  b <= p  and  p + 2 <= b + 65507.
Admitted first-byte address:      p = b - 1.
Enclosing-object address:         p = B + 3, inside the same struct.
```

3. Exact instrumented certificate cases

Both selectors use zero-based disjunct index 1 and empty inline context. `sp0` is the proof environment stack anchor; the three builds have different frame layouts. `both` is the repository name for ASan+UBSan. [P2] [P3]

Item	ASan	ASan+UBSan
Unit / entry	752 / 0x8d0f0	845 / 0x8c638
Cut / disjunct	0x8d348 / 1	0x8c93c / 1
Field start b	sp0 - 1326956	sp0 - 1327020
First-byte destination p	sp0 - 1326957	sp0 - 1327021
Register facts	x14 = b; x18 = -1; x14 + x18 = p	x22 = b; x25 = p
First byte-store PC	0x8d38c: sb x10, 0(x21)	0x8c97c: sb x10, 0(x25)
Instruction word	0x00aa8023	0x00ac8023
Containing write interval (relative to sp0)	[-1326957, -1326932)	[-1327021, -1327008)
Shadow offset relative to sp0/8 + 0x7fff8000	-165870, byte value 0	-165878, byte value 0
Later cut / recorded slot	0x8d3e8 / offset -1326957, width 1	0x8c9e8 / offset -1327021, width 1

In ASan, the selected cut is before the addition that forms x21. The theorem computes the destination as machine-word addition x14+x18. In ASan+UBSan, x25 already holds the destination. The combined build's extra recorded constraints therefore do not restore the field lower bound.

Static instruction inspection places the combined build's pointer checks at 0x8c8b4-0x8c8d4, before its selected cut. The later ASan shadow tests examine zero in the supplied model. These instruction associations are source/binary audit evidence; the new Rocq statements are state-existence theorems, not complete small-step traces through the C operation.

Certificate reference chains are ASan 914 -> 754 at call PC 0x86ae8, then 754 -> 752 at 0x8e380; and ASan+UBSan 1034 -> 847 at 0x86610, then 847 -> 845 at 0x8d6d0. These references identify the reviewed call contexts, without establishing that an input reaches the selected disjunct.

The formal claim

```
There exist valuation nu, registers R and memory M such that:
  original Gamma_2(case, nu, R, M)
  and rank(case, nu) >= 0
  and destination(R) = b - 1 < b
  and store_ok(M, destination(R), 1) = true
  and ASan shadow(destination(R)) = 0.
Therefore Gamma_2(case) does not entail b <= destination.
```

The two final statements are `asan_decoded_case_admits_buf_minus_one` at `reproduction/proof/Complete.v:56` and `both_decoded_case_admits_buf_minus_one` at line 127. Each selects the unit, full state and rank from the original compressed certificate and uses the original `unit_params2/genv` parameters and the SHA-pinned binary images.

4. The exact machine-checked statements

These are literal excerpts from the successful audit sources, not schematic pseudocode. `asan_buffer` and `both_buffer` are the field starts; `asan_destination` is the machine addition x14+x18, and `both_destination` is register x25. The definitions and all witnesses are carried in `reproduction/proof/`.

ASan: reproduction/proof/Complete.v:56-68

```
56  Theorem asan_decoded_case_admits_buf_minus_one :
57   exists U nu regs mem,
58    find (fun u => Z.eqb (u_vid (u1 u)) 752) asan_decoded = Some U /\
59    select_case asan_decoded 752 0x8d348 = Some asan_case /\
60    select_rank asan_decoded 752 0x8d348 = Some asan_rank /\
61    gamma2 SEGSA (unit_params2 (genv_asan true) U) asan_E mem
62      (load_cstate2 (unit_params2 (genv_asan true) U) asan_case 0x8d348 []) nu regs mem /\
63    0 <= eval nu asan_rank /\
64    bv_unsigned (asan_destination regs) = asan_buffer-1 /\
65    bv_unsigned (asan_destination regs) < asan_buffer /\
66    store_ok SEGSA mem (asan_destination regs) 1 = true /\
67    rdn SEGSA mem (wordZ (0x7fff8000 + bv_unsigned (asan_destination regs)/8)) 1
68      = Some [Z_to_bv 8 0].
```

ASan+UBSan: reproduction/proof/Complete.v:127-139

```
127  Theorem both_decoded_case_admits_buf_minus_one :
128   exists U nu regs mem,
129    find (fun u => Z.eqb (u_vid (u1 u)) 845) both_decoded = Some U /\
130    select_case both_decoded 845 0x8c93c = Some both_case /\
131    select_rank both_decoded 845 0x8c93c = Some both_rank /\
132    gamma2 SEGSB (unit_params2 (genv_both true) U) both_E mem
133      (load_cstate2 (unit_params2 (genv_both true) U) both_case 0x8c93c []) nu regs mem /\
134    0 <= eval nu both_rank /\
135    bv_unsigned (both_destination regs) = both_buffer-1 /\
136    bv_unsigned (both_destination regs) < both_buffer /\
137    store_ok SEGSB mem (both_destination regs) 1 = true /\
138    rdn SEGSB mem (wordZ (0x7fff8000 + bv_unsigned (both_destination regs)/8)) 1
139      = Some [Z_to_bv 8 0].
```

`SEGSA` and `SEGSB` are the static images regenerated from the SHA-pinned instrumented ELF files. `store_ok` here checks permission for the first byte; this is enough to exhibit the missing lower bound. These existence statements do not assert a small-step execution to, or through, the write.

5. Why the internal proof permits the neighboring byte

The complete extended interpretation `gamma2` includes the base invariant, environment, runtime memory, fixed symbols, division definitions, all regions, liveness, symbol bounds, globals, flags and memoized facts. The new witnesses satisfy this entire predicate. [P4] [P5]

Complete extended concretization predicate gamma2 [P4]

```
414    Definition gamma2 (mem0 : word -> option byte) (A : vstate2) (nu : val) (regs : bvmap 5 word)
     (mem : word -> option byte) : Prop :=
415      gamma SEGS P1 E1 (zpatch mem0 mem) (v1 A) nu regs mem
416      /\ env_ok2 /\ MC2 mem0 mem /\ gamma_fix2 E nu
417      /\ gamma_divdef nu A
418      /\ (forall r, In r (vs_rgn A) -> region_ok nu A mem r)
419      /\ live_ok nu A
420      /\ state_below2 A
421      /\ globs_ok2 A
422      /\ flags_inv A nu mem
423      /\ memo_ok nu mem A.
```

For these ordinary SP0-relative accesses, the extended store machinery falls back to the ordinary stack store checker. [P7]

Extended store falls back to ordinary store checker [P7]

```
494  Definition x_store2 (P : vparams2) (A : vstate2) (sz : msize) (rs1 rs2 : Z) (imm : Z) : vresult2
     :=
495    let n := msize_z sz in
496    match addr_of (v1 A) rs1 imm with
497    | None => VR2_fail "store via Top address"
498    | Some addr =>
499        match classify2 A addr n true with
500        | None => lift1c A (x_store (p1 P) (v1 A) sz rs1 rs2 imm)
501        | Some (C2Fail m) => VR2_fail m
502        | Some C2Dead => VR2_next []
503        | Some (C2Rgn r off lo hi) => store_rgn A r off lo hi n (get_reg (v1 A) rs2)
504        end
```

The ordinary classifier requires the access to be inside the stack window and the unit write set. It does not attach the `buf` field's start address to this pointer. The decoded write intervals in the preceding table explicitly include the byte before `buf`. [P6] [P8]

Ordinary stack-window and write-set checks [P6]

```
 52  Definition classify (P : vparams) (A : vstate) (addr : lin) (n : Z) (is_store : bool) : cres :=
 53    let cs := lcoef addr SP0 in let cd := lcoef addr DATA in
 54    if Z.eqb cs 1 && Z.eqb cd 0 then
 55      let off := lsub addr (lsym SP0) in
 56      (* [comment removed for export] *)
 57      if negb (qge A off (- vp_frame P)) then
 58        (if bnd_dead A off then CDead else CFail "stack access below window")
 59      else if negb (qle A off (- n)) then
 60        (if bnd_dead A off then CDead else CFail "stack access above sp0")
 61      else
 62        let b := bnd A off in
 63        match b with
 64        | (Some lo, Some hi) =>
 65            if itv_empty b then CDead
 66            else if is_store && negb (in_ranges (vp_wstack P) lo (hi - lo + n)) then CFail "store
     outside unit write set"
 67            else COk (RStack off lo hi)
 68        | _ => CFail "stack offset unbounded"   (* [comment removed for export] *)
```

The extra fake-frame, borrowed-argument, flag and shadow regions are all accounted for in the witnesses. Their existence does not supply the missing bound b <= p at this access. The result follows from the complete predicate, rather than from inspecting only the exported agreement statement.

6. How this matches Section 4 of the paper

The current Section 4, "An Example", uses `struct S { char buf[20]; unsigned char op; }` and a fill operation that may cross from `buf` into `op`. The modified opcode still selects an in-bounds element of a 256-entry handler table. [M1] The section distinguishes mapped/permitted enclosing storage from the bounds of its finer-grained objects. [M2]

Feature	Section 4	Coturn finding
Field whose bounds matter	20-byte buf	65507-byte buf
Neighbor in the same aggregate	op, following buf	channel[3], preceding buf
Crossed boundary	Upper boundary: forward overflow	Lower boundary: underflow
Containing storage	Same allocated struct	Same stack-resident struct; allowed stack/write-set extent
Sanitizer distinction	No effective field boundary in the example	Checked model has zero ASan shadow at p; combined-build invariant also admits p
What is established here	The paper discusses a constructed reachable overwrite and dispatch consequence	A countermodel to field-bound entailment by the selected internal invariants

The shared point is a subobject boundary that is absent from the effective proof obligation. An address can remain within permitted enclosing storage while violating the bounds of the particular array field from which the pointer was formed. A two-byte write starting one byte before `buf` already violates the field lower bound, even if only its first byte targets the neighboring field.

For UBSan, the paper can use a narrow, checked claim: in this particular ASan+UBSan build and certificate case, the complete retained constraints admit the already formed out-of-field pointer. This does not require a universal claim about every UBSan configuration or every pointer-decay optimization.

The coturn evidence establishes the field-boundary distinction. It does not establish a reachable coturn attack, a changed channel byte, a dispatch effect, or a false published agreement theorem.

7. Material ready to reuse in the paper

Compact paragraph

Coturn's client-message proofs provide a concrete instance of this field-boundary gap. Its `stun_buffer` places a four-byte `channel` array immediately before a `buf` array. The routine `stun_attr_add_str` forms `attr_start = buf + clen` and writes two bytes there. We machine-checked abstract states admitted by both the ASan and ASan+UBSan internal cut-invariants in which that address is `buf - 1`, the storage of `channel[3]`. These states satisfy the complete original invariant, including its memory permissions and shadow facts. Thus the existing invariants do not establish that this access stays inside the array field. This is a result about the strength of the internal invariants; it does not establish that a covered input reaches the state or changes the program's behavior.

Compact listing (explicitly abridged)

Paper illustration: macros expanded, omitted code marked; not a standalone program.

```
typedef struct _stun_buffer {
  uint8_t channel[4];
  uint8_t buf[65507];
  /* len, offset, coffset omitted */
} stun_buffer;

/* Inside stun_attr_add_str(uint8_t *buf, ...): */
const int clen =
    stun_get_command_message_len_str(buf, *len);
/* newlen calculation and padding omitted */
if (newlen >= MAX_STUN_MESSAGE_SIZE)
  return false;
uint8_t *attr_start = buf + clen;
/* update message length */
turn_write_u16(attr_start, attr);
```

Use `paper-snippets.tex` for a self-contained LaTeX insertion containing this paragraph and listing. `paper-verbatim.tex` contains the unabridged source excerpts with the pinned source references. Original source files and licence headers are preserved under `upstream/coturn/`; numbered excerpt files under `code/` retain the original copyright/licence header.

Technical formulation

Let b be the start of `stun_buffer.buf` and p the destination of the first lowered byte store for `turn_write_u16(attr_start, attr)`. Field-level safety requires at least `b <= p`. For ASan unit 752 at cut `0x8d348`, disjunct 1, and ASan+UBSan unit 845 at cut `0x8c93c`, disjunct 1, the new Rocq lemmas construct valuations, registers and memory satisfying the complete original `gamma2` predicate while `p = b - 1`, `store_ok(p, 1) = true` and the corresponding ASan shadow byte is zero. The cases, ranks and relevant unit parameters are checked against the original compressed certificates with the repository's decoder. Consequently, these cut-invariants do not entail the field lower bound. This countermodel does not establish a concrete execution from the harness input, or refute the published agreement theorem.

The paper's current `\repo` macro points to older proof commit 98e995d4. The supplied snippets use their own `\coturnproofrepo` macro pinned to a6839bb9. Do not silently cite the new finding through the older link macro. The supplied bibliography file provides entries for the pinned proof repository and coturn source.

8. Plain-build evidence and the bzip2 result

Plain coturn case: a distinct evidence level

The plain-build investigation found unit 166, entry 0x18898, cut 0x18994, context [], disjunct 0. Its field start is sp0-65628 and x24 is sp0-65629. The instruction at 0x18998 is `sb x10, 0(x24)` (word 0x00ac0023); the next stored cut at 0x189ac records a one-byte slot at that offset. [P1]

The unit write set contains [-65629,-65585) and [-120,-112), relative to sp0. The selected pure constraints admit s21=12, s23=4, s27=20, s28=8, sp0=0x7fff80000000, DATA=0x400000 and SIZE=1. The corrected congruence for s27 is 4 modulo 8. Slot consistency, pure constraints, instruction bytes and range inclusion were checked by Python.

A certificate reference chain connects root 249 to unit 170 at call PC 0x14c90, then to unit 166 at 0x192f0. These are proof/call references, not a reachability proof for the selected abstract disjunct. This investigation did not add a full original-gamma Rocq existence theorem for the plain variant. The complete decoded data and earlier report are in `evidence/plain-witness.json` and `evidence/plain-audit.txt`.

bzip2: the initially proposed example did not survive internal review

The bzip2 review used the same proof commit and source pin f5dfc96063fef2d9c5446d0733867d899ef9b942. It inventoried 115 fields in the core structs and 24 inline arrays, checked their offsets, lengths and element widths against the RV64 layout, and inspected the associated bounded-access proof families. No concrete omitted field-bound obligation was identified in that source review.

This is a manual-review result, not a newly machine-checked universal C-subobject safety theorem. The completed harness compresses and then decompresses its own output; standalone arbitrary-input decompression and file-I/O API histories have different coverage. Coarsened saved fields concern persistent value/session contracts, not an identified field-overflow counterexample. The full inventory is preserved as `evidence/bzip2-field-bounds-audit.json`.

The bzip2 result is useful methodological context: omission of a property from an exported theorem does not by itself establish that the internal proof omits it. Coturn supplies the stronger evidence because its selected complete internal predicates have explicit checked countermodels.

9. bzip2: core inline-array inventory

All 24 core inline arrays in the manual audit are listed below. Sizes and offsets are bytes in the audited RV64 layout. The JSON inventory also records the other 91 scalar/pointer fields and the exact bounded-access proof-family locations for every field. No row is a newly replayed universal field-safety theorem.

Array field	Offset	Size
EState.inUse	128	256
EState.unseqToSeq	384	256
EState.mtfFreq	672	1032
EState.selector	1704	18002
EState.selectorMtf	19706	18002
EState.len	37708	1548
EState.code	39256	6192
EState.rfreq	45448	6192
EState.len_pack	51640	4128
DState.unzftab	68	1024
DState.cftab	1096	1028
DState.cftabCopy	2124	1028
DState.inUse	3196	256
DState.inUse16	3452	16
DState.seqToUnseq	3468	256
DState.mtfa	3724	4096
DState.mtfbase	7820	64
DState.selector	7884	18002
DState.selectorMtf	25886	18002
DState.len	43888	1548
DState.limit	45436	6192
DState.base	51628	6192
DState.perm	57820	6192
DState.minLens	64012	24

The per-field evidence is in `evidence/bzip2-field-bounds-audit.json`. It links to the same current proof commit a6839bb9. The completed bzip2 proof is a source-review contrast to the checked coturn countermodels, rather than a positive missing-field-bound finding.

10. Decoder correction, verification and reproduction

Correction to the earlier extraction

The first Python decoder parsed a congruence residue as unsigned. The upstream decoder uses the signed parser for the residue: the format is symbol, unsigned modulus, signed residue. [P9] Rocq's equality check against the original certificate exposed the error.

Item	Earlier extraction	Corrected evidence
Instrumented SP0 congruence	Residue 32 modulo 32	Residue 16 modulo 32
Instrumented SP0 witness	0x7fff80000000	0x7fff80000010
Plain s27 congruence	Residue 8 modulo 8	Residue 4 modulo 8
Plain s27 witness	24	20

The decoder, decoded data and published witnesses were corrected. The field addresses remain one byte before `buf`, so the final finding is unchanged. The final instrumented source literals and ranks are checked by the original Rocq decoder; the Python translation is not part of the trusted formal check.

Recorded machine check

The original fresh build passed all 88 sources in 166.78 seconds on 1 November 2026 (finished 04:11:52 UTC), using the release-pinned Rocq 9.2 checker with OCaml 4.14.3. The 75 copied coturn dependencies, 11 audit sources and two generated image modules were recompiled. Rocq and LPS libraries are supplied by the pinned checker image. `Print Assumptions` for the final theorems lists 21 allowed Rocq array, integer and string primitives; no new axioms, admitted lemmas or unchecked casts were introduced. The source SHA-256 values match the carried build record.

Rocq checks exact equality of the selected state and rank with the original compressed data. It also checks relevant unit-parameter fields against the decoded unit. The witnesses construct an environment, valuation, registers and concrete memory for every clause of the complete original gamma2 interpretation. `gamma_params_exact` proves that discarding unused static-table fields during computation preserves the entire predicate, and the final theorem statements restore the original parameters.

Reproduce the instrumented results

Requires Docker and the pinned checker image already available locally.

```
cd reproduction
./run.sh > check.log 2>&1
# Inspect result.json and assumptions.txt.
```

The container has networking disabled. The driver checks the binary and upstream-library hashes, regenerates both static images, recompiles all 88 sources and checks the final assumptions against the copied upstream allowlist. Neither RISC-V coturn binary is executed. On an ARM Mac, Docker runs the release's amd64 checker image through emulation. The complete proof sources, pinned ELF inputs and build records are included.

The checker image is `ghcr.io/theorem-labs/oss-fuzz-theorems-checker` at digest `sha256:cc9ae43a9f4b0dd59d7c8d30727d36e039bfe72a2a9f9c4a99c68e413037e902`. The release pins LPS commit `f1006fe8178338a499b5e5c41be83006c4edd058`.

11. Evidence register and claim boundaries

Artifact	What it supplies
reproduction/proof/Certificate.v and Rank.v	Exact equality with original selected state and rank.
reproduction/proof/Model.v	Environment, memory, registers and exact gamma-parameter reduction.
reproduction/proof/Pure.v	Intervals, congruences, fixed symbols, division definitions and ranks.
reproduction/proof/Regions.v	Every loaded region: cells, permission, kind and ownership.
reproduction/proof/Ambient.v	Environment, pristine memory, runtime context and shadow data.
reproduction/proof/Structural.v and Flags.v	Registers, slots, NUL, liveness, well-formedness and flag invariants.
reproduction/proof/Complete.v:56, :127	Final existence statements over original decoded units and image segments.
evidence/original-rocq-result.json / check.log	Original fresh build, source and binary hashes, assumptions result.
evidence/instrumented-witnesses.json	Full selected decoded states, valuations and audit metadata.
PROVENANCE.json / MANIFEST.sha256	Source pins, extracted-code records and package integrity.

Claims suitable for the paper

• The selected complete coturn cut-invariants do not entail the lower bound of the buf array field.

• Both instrumented variants have checked invariant models with destination = buf - 1, writable memory there and a zero ASan shadow byte.

• This is an internal proof-strength result, stronger evidence than absence of field bounds in an exported theorem statement alone.

• The field-boundary distinction matches the within-aggregate point illustrated in Section 4.

Claims not established by this investigation

• A harness input reaches either abstract state, or makes this helper return -1 in the reviewed caller context.

• The channel byte changes, is attacker-controlled, or affects later behavior.

• A complete execution through the two-byte C write, a vulnerability, or an exploit.

• A counterexample to the published agreement theorem, or a universal result about every coturn access, UBSan configuration, or source-level C object.

The formal result is a non-entailment statement about particular accepted internal predicates. The C correspondence and source layout explain which source-field obligation is absent. These two evidence layers are recorded separately so the paper can state the conclusion at its demonstrated strength.

12. Pinned source references

Source identifiers below link to immutable repository revisions. The original files and source excerpt hashes are included in PROVENANCE.json. New audit lemmas are local artifacts in the reproduction directory; they are not presented as upstream-merged changes.

C1 stun_buffer declaration
src/apps/common/stun_buffer.h:45-51

C2 Message/array capacity constants
src/ns_turn_defs.h:221-222

C3 STUN header and channel-header constants
src/client/ns_turn_msg_defs.h:46-47

C4 Struct wrapper passes its buf field
src/apps/common/stun_buffer.c:181-184

C5 Channel-bind request builder
src/client/ns_turn_msg.c:1300-1327

C6 Address attribute builder calls stun_attr_add_str
src/client/ns_turn_msg.c:1705-1733

C7 Signed message-length result and -1 failure
src/client/ns_turn_msg.c:523-535

C8 Complete stun_attr_add_str body
src/client/ns_turn_msg.c:1666-1703

C9 Two-byte write helper
src/ns_turn_defs.h:145-148

P1 Plain compressed certificate
proofs/coturn/FuzzStunClient/lib/VCertS_src0_p01.v:5-5

P2 ASan compressed certificate (dictionary: p00)
proofs/coturn/FuzzStunClient/lib/VCertS_asan_p01.v:5-5

P3 ASan+UBSan compressed certificate (dictionary: p00)
proofs/coturn/FuzzStunClient/lib/VCertS_both_p01.v:5-5

P4 Complete extended concretization predicate gamma2
proofs/coturn/FuzzStunClient/lib/VGamma2.v:414-423

P5 Base concretization predicate gamma
proofs/coturn/FuzzStunClient/lib/VGamma.v:89-93

P6 Ordinary stack-window and write-set checks
proofs/coturn/FuzzStunClient/lib/VStepMem.v:52-68

P7 Extended store falls back to ordinary store checker
proofs/coturn/FuzzStunClient/lib/VStepMem2.v:494-504

P8 Construction of per-unit parameters
proofs/coturn/FuzzStunClient/lib/VDriver2.v:43-48

P9 Congruence residue uses signed parser
proofs/coturn/FuzzStunClient/lib/VCertDecode.v:125-126

P10 Dictionary-backed extended state decoder
proofs/coturn/FuzzStunClient/lib/VCertDecode2.v:132-150

M1 Section 4 interpreter example
blog/what-did-they-actually-prove/evidence/paper/main.tex:193-222

M2 Section 4: mapped storage, ASan, UBSan and agreement
blog/what-did-they-actually-prove/evidence/paper/main.tex:277-306

Binary identity

`FuzzStunClient.asan`: `99443988bd9bcd3087fe619757c7dff2a6b198583ad959307bc34edcd9b44ef8`

`FuzzStunClient.asan-ubsan`: `4fd7bfa4d6ebf2583737cfa6cc09ce0f3344066b8f8c92d899ff91843d1d9f69`

The plain binary SHA-256 is `087641c6fcee7a7c11340e42b08651160429ec9d1ec42724a9560455214ff81b`. The instrumented binary archive SHA-256 recorded during acquisition is `35b053abc428e398e6dc25d27c7eb1ad8f3845dabef4a881ba57c09e4e4910ca`. Upstream build records and licences are carried in `reproduction/upstream-provenance/`.